



First steps to Python for Humanities & Social Sciences

About

Who is this course for?

This course aims to provide a starting point for students and researchers from the Humanities and Social Sciences interested in learning Python. This course is for beginners. After completing the four modules, you can continue your learning by participating in other open online courses, joining a Carpentries workshop, working through available open educational resources, or joining a local community of practice.

This introductory course is offered as part of the [ESCALATOR programme](#), an initiative of the [South African Centre for Digital Language Resources](#).

Workshop format

Instructors will work through software installation and the very first steps of using Python with open and free training resources. This is a hands-on code-along course, meaning candidates will work on their computers and follow along with the instructor as they work through the material.

At the end of this course, you will be able to:

- Recognise terminology used in programming
- Open the programming environment on your computer
- Run some basic code
- Load and inspect tabular data
- Know where to find help to continue on the journey
- Identify next steps

Funding

These lessons form part of the [ESCALATOR programme](#) developed by the [South African Centre for Digital Language Resources](#) (SADiLaR). SADiLaR is a national centre supported by the Department of Science and Innovation (DSI).

Data

The data used in this course is the NCHLT English Text Corpora that you can download from the SADiLaR Resource Catalogue:

<https://repo.sadilar.org/handle/20.500.12185/301>

This data is a collection, consisting of a clean corpus, lexicon, frequency list and named-entity lists developed during the *NCHLT Text* project.

Contributors

- Anelda van der Walt
- Bianca Peterson
- Shani Le Roux
- Nomalungelo Maphanga

Licence

This course is licenced as CC-BY 4.0.

The data is licenced as [Creative Commons Attribution 2.5 South Africa License](#) (CC BY 2.5).

Programming software installations

Install Anaconda and Jupyter notebooks

Anaconda and Jupyter Notebooks are two separate pieces of software:

- Python is a general-purpose programming language that supports the rapid development of data analytics applications. The word “Python” is used to refer to both the programming language and the tool that executes the scripts written in Python language.

- Anaconda is a distribution of Python that includes a lot of its popular packages, such as the IPython console, Jupyter Notebook, and Spyder Integrated Development Environment (IDE).
- Jupyter Notebook is an open-source web application that allows you to create and share documents that combine code, graphs, and narrative text.

If you don't already have Anaconda and Jupyter Notebook installed, follow the instructions for your operating system below. You have to install Anaconda before you install Jupyter Notebook.

Anaconda

Windows

1. Open <https://www.anaconda.com/products/individual> in your web browser.
2. Download the Anaconda Python 3 installer for Windows.
3. Double-click the executable and install Python 3 using the recommended settings. Make sure that Register Anaconda as my default Python 3.x option is checked – it should be in the latest version of Anaconda.
4. Verify the installation: click Start, search and select **Anaconda Prompt** from the menu.

Here is a video tutorial for further assistance:

<https://youtu.be/xxQ0mzZ8UvA?feature=shared>

MacOS

1. Visit <https://www.anaconda.com/products/individual> in your web browser.
2. Download the Anaconda Python 3 installer for MacOS. These instructions assume that you use the graphical installer .pkg file.
3. Follow the Anaconda Python 3 installation instructions. Make sure that the install location is set to “Install only for me” so Anaconda will install its files locally, relative to your home directory. Installing the software for all users tends to create problems in the long run and should be avoided.
4. Verify the installation: click the Launchpad icon in the Dock, type *Terminal* in the search field, then click Terminal.

Here is a video tutorial for further assistance:

<https://youtu.be/TcSAln46u9U?feature=shared>

Linux

Note that the following installation steps require you to work from the terminal (shell). If you run into any difficulties, please request help before the workshop begins.

1. Open <https://www.anaconda.com/products/individual> in your web browser.
2. Download the Anaconda Python 3 installer for Linux.
3. Install Anaconda using all of the defaults for installation:
 - Open a terminal window.
 - Navigate to the folder where you downloaded the installer.
 - Type `bash Anaconda3-` and press `Tab`. The name of the file you just downloaded should appear.
 - Press `Return`.
 - Follow the text-only prompts. When the license agreement appears (`--more--` will be present at the bottom of the screen) press `Spacebar` until you see the bottom of the text. Type `yes` and press `Return` to approve the license. Press `Return` again to approve the default location for the files. Type `yes` and press `Return` to prepend Anaconda to your `PATH` (this makes the Anaconda distribution your user's default Python).
4. Verify the installation: this depends a bit on your Linux distribution, but often you will have an Applications listing in which you can select a Terminal icon you can click. Alternatively, type `jupyter notebook` and press `Return`. A Jupyter notebook should open in your web browser.

Zoom installation

Please ensure you have the latest version of Zoom installed to get the best out of our virtual meetups.

Download Zoom from <https://zoom.us/download>.

If you are experiencing any challenges, please get in touch by submitting a comment at the bottom of this page. We will respond there.

Weekly Zoom session

Join Zoom Meeting

Please ensure you join the sessions at least 5 minutes earlier, just in case your internet connection takes some time.

To join the weekly Zoom training sessions, please click on the link, or use the meeting details below:

Week 1: Getting Started

1. Understand the different elements in the programming environment
2. Describe how programming can be applied in specific contexts in humanities or social sciences
3. Open the relevant software to use on your computer
4. Be familiar with the user interface
5. Set up a computational project according to best practices

Applications of programming

Applications of Python programming

The Python programming language is widely used in social sciences and humanities for various data analysis, visualisation, and research purposes.

Here are some real-world examples of its applications:

Text Analysis and Natural Language Processing (NLP)

Researchers in humanities and social sciences often use Python to analyse large text corpora.

For instance, analysing historical documents, literature, or social media data to extract patterns, sentiment, themes, and other linguistic features.

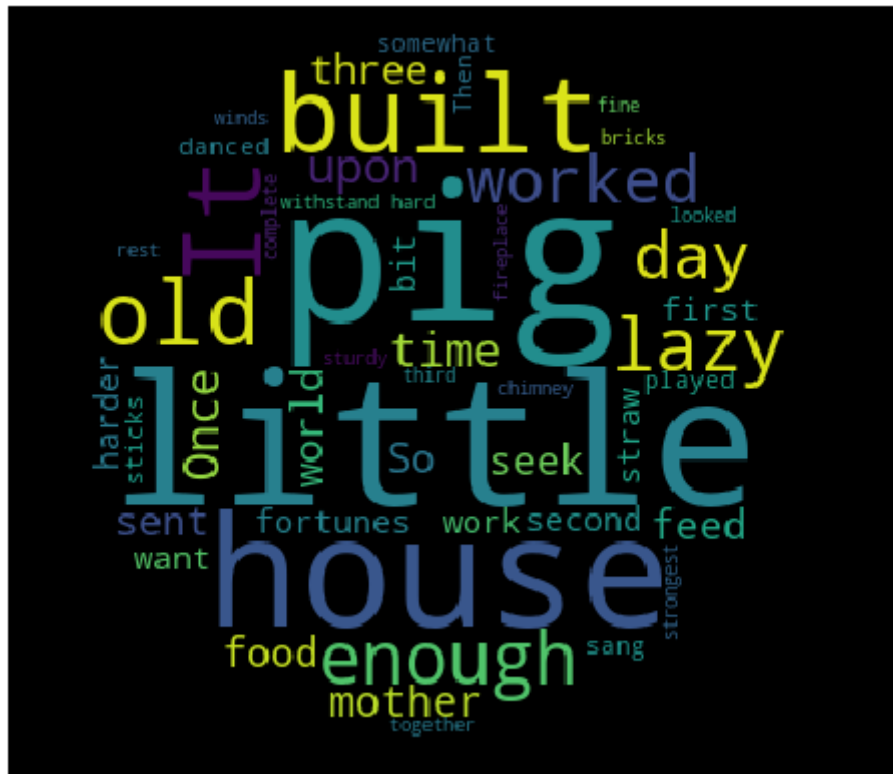


Example: A historian might use Python to analyse a collection of letters from a particular time period to uncover trends in language use, sentiment, and societal changes.

For a comprehensive tutorial on how to preprocess text, analyse text and create visualisations based on the text analysis, see the following tutorials:

- [Natural Language Processing With Python's NLTK Package](#)

- Natural Language Processing (NLP) with Python—Tutorial



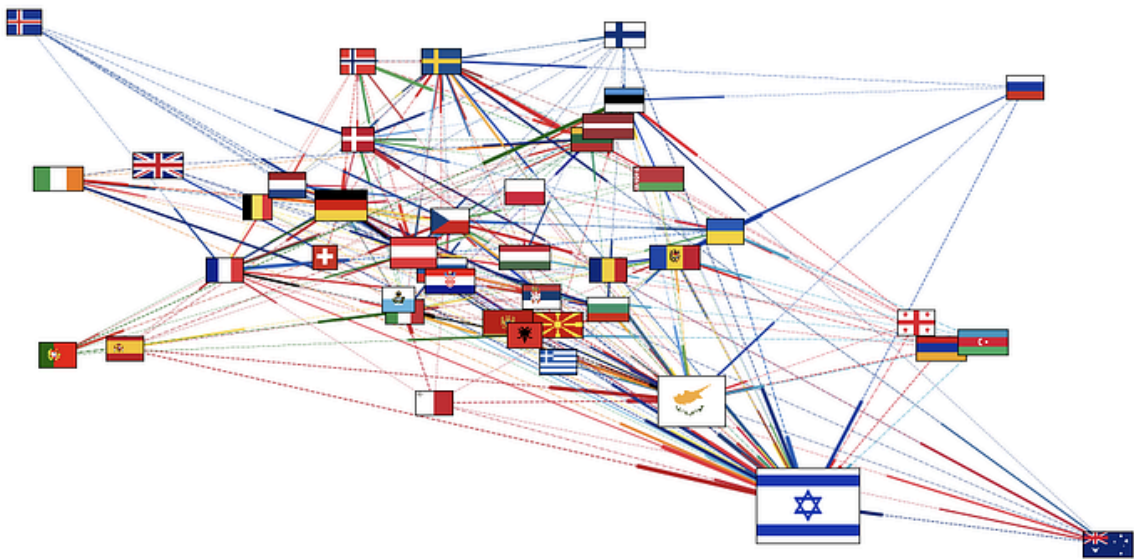
Social Network Analysis

Python is frequently used to study social networks, both online and offline. Researchers can analyse patterns of interaction, influence, and information flow within networks.

 Example: Sociologists might use Python to analyse connections between individuals in a community and study how information spreads through a social network.

The following step-by-step tutorial shows [Social Network Analysis: From Graph Theory to Applications with Python](#).

Eurovision 2018 Final Votes



Data Visualisation

Python's libraries like Matplotlib, Seaborn, and Plotly are commonly used to create visualisations that help researchers present their findings effectively.

 Example: An anthropologist might use Python to create interactive visualisations showing migration patterns and cultural interactions between different regions over time.

The [Introduction to Cultural Analytics & Python](#) online book has a step-by-step tutorial for various visualisations, including bar graphs, network analyses and heatmaps, such as the one below.

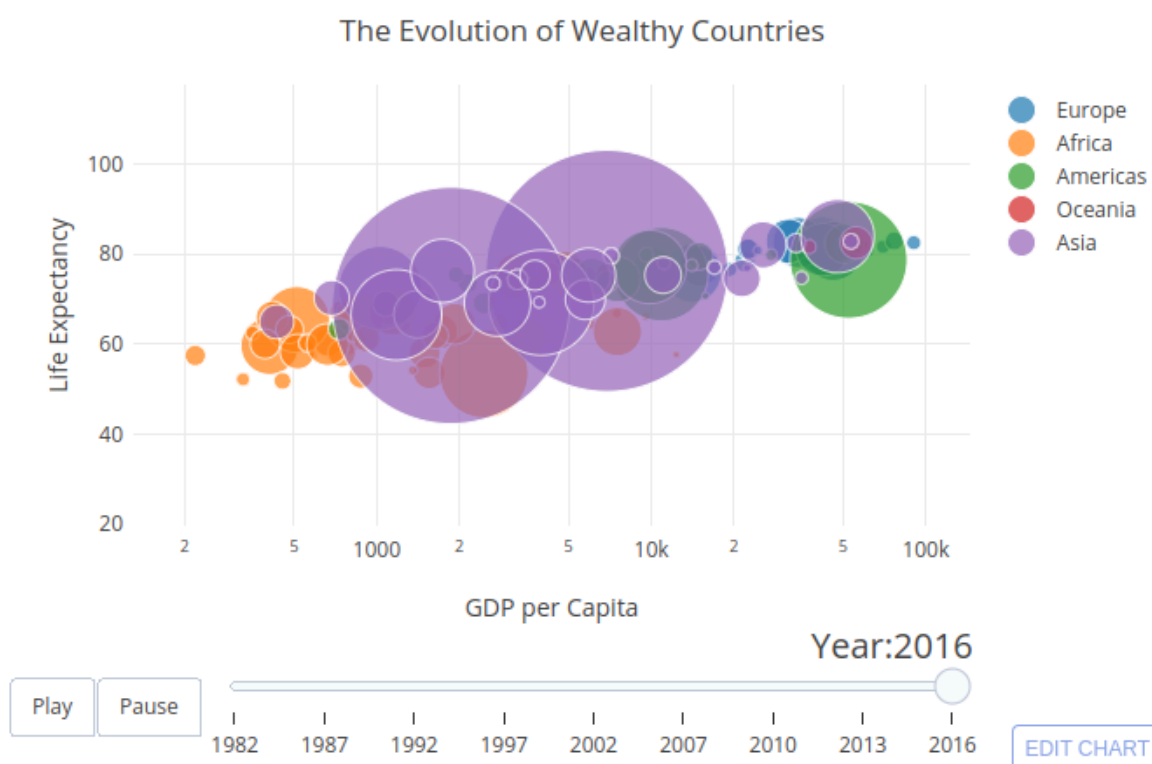


Quantitative Analysis

Python is used extensively for statistical analysis and modelling in social sciences. Libraries like NumPy, Pandas, and Statsmodels facilitate data manipulation, hypothesis testing, and regression analysis.

 Example: Economists might use Python to analyze economic data, run regression models to understand factors influencing economic growth, and visualize their results.

The following step-by-step tutorial shows how to [visualise economic data using Plotly](#).

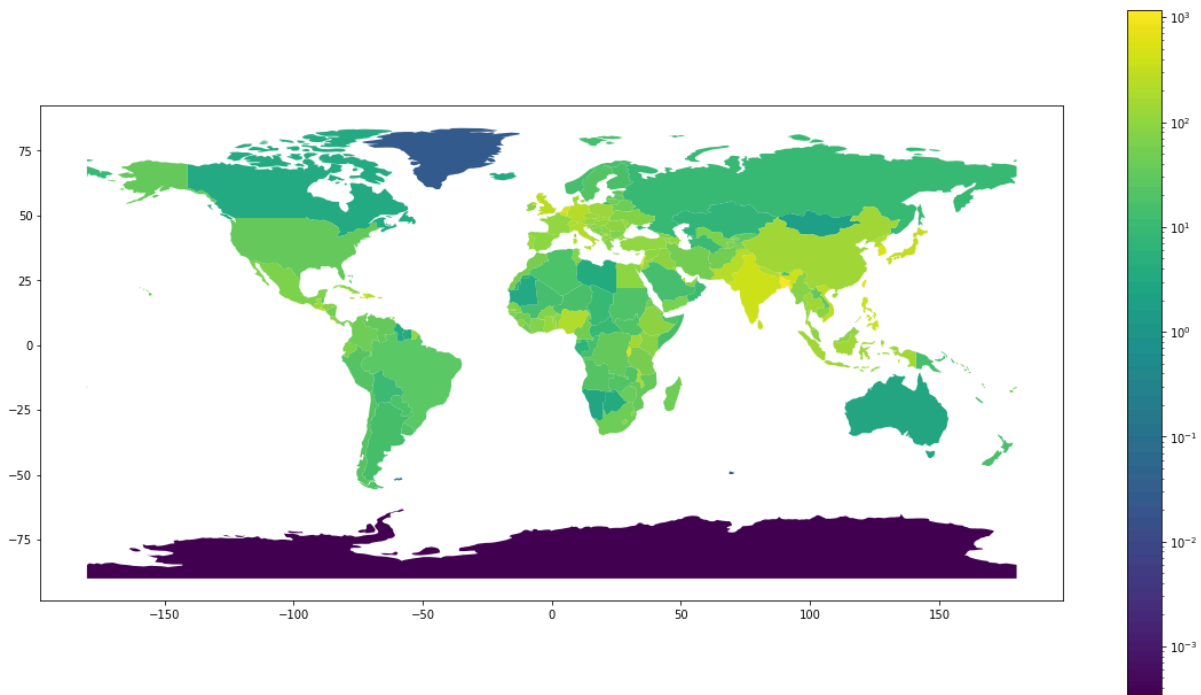


Geospatial Analysis

Python's libraries like GeoPandas and Folium allow researchers to work with geospatial data. This is particularly useful for analysing geographic patterns and visualising spatial relationships.

 Example: Urban planners could use Python to analyse patterns of land use, transportation routes, and demographic trends within a city.

The following step-by-step tutorial demonstrates how to [analyse geospatial data in python using GeoPandas and Shapely](#).

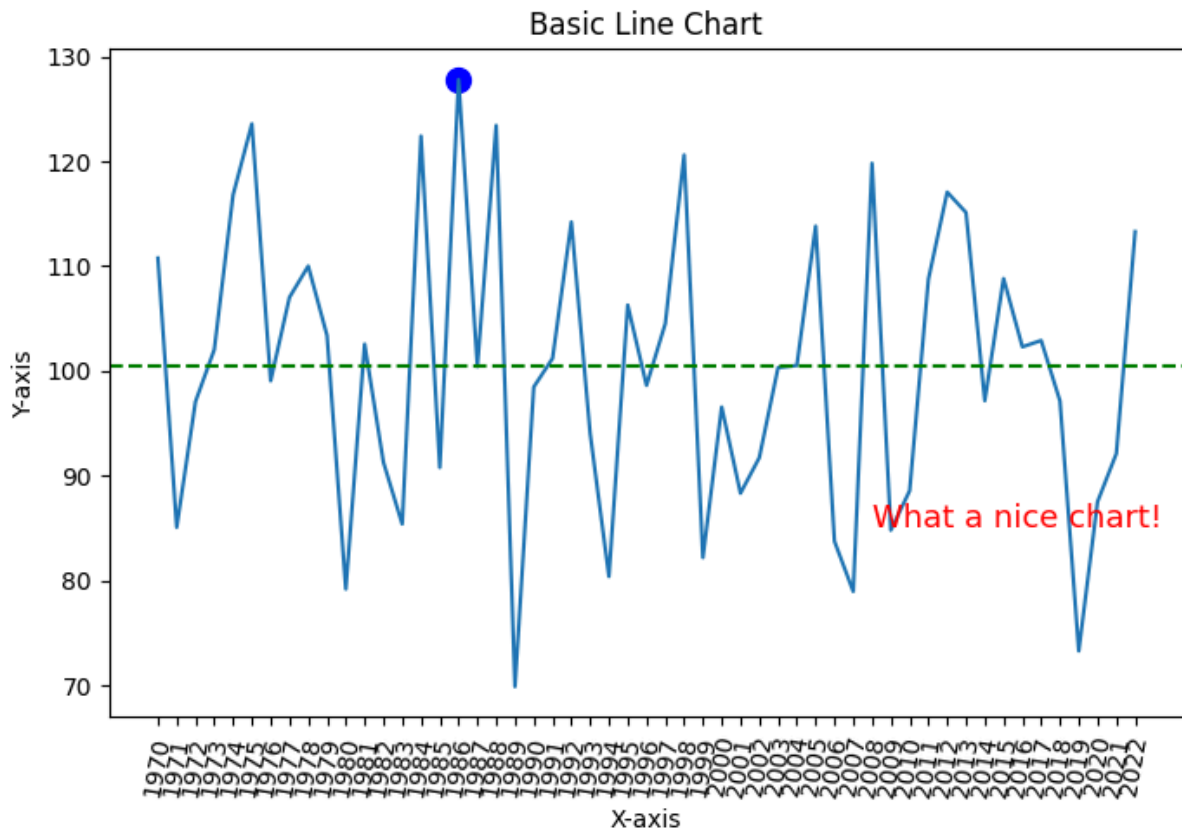


Digital Humanities

Python is integral to many digital humanities projects, where researchers combine traditional humanistic research with computational methods.

 Example: A linguist might use Python to process and analyse a large corpus of historical texts to study changes in language usage and cultural evolution over time.

The following step-by-step tutorial demonstrates [how to make a basic line chart with annotations](#).

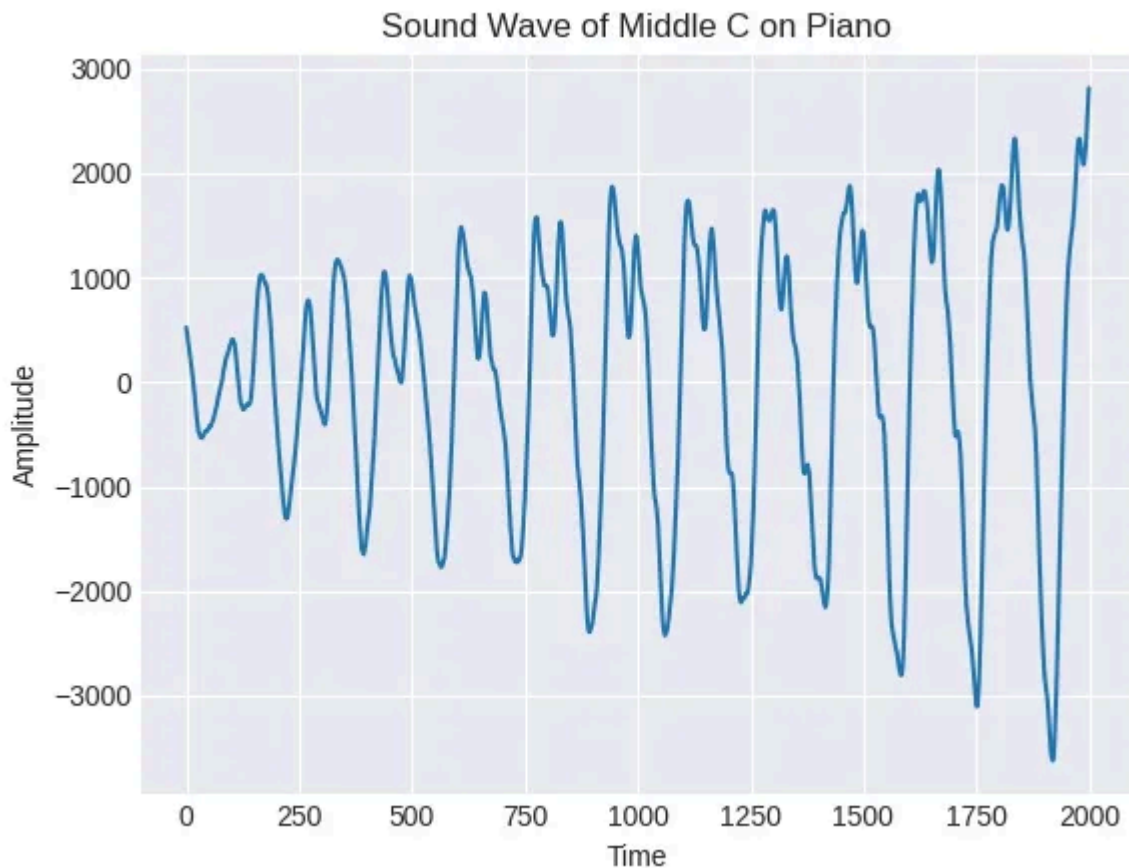


Cultural Analytics

Researchers use Python to analyse various cultural artefacts such as art, music, and literature. This can involve extracting patterns, themes, and styles.

 Example: A musicologist could use Python to analyse musical scores and compositions to uncover trends in musical structure and genre evolution.

The following tutorial demonstrates how you can [visualise musical concepts using python](#).



These examples showcase how Python's flexibility and rich ecosystem of libraries make it a valuable tool for researchers in social science and humanities fields. There are many more free tutorials online!

Open the relevant IDE software on your computer

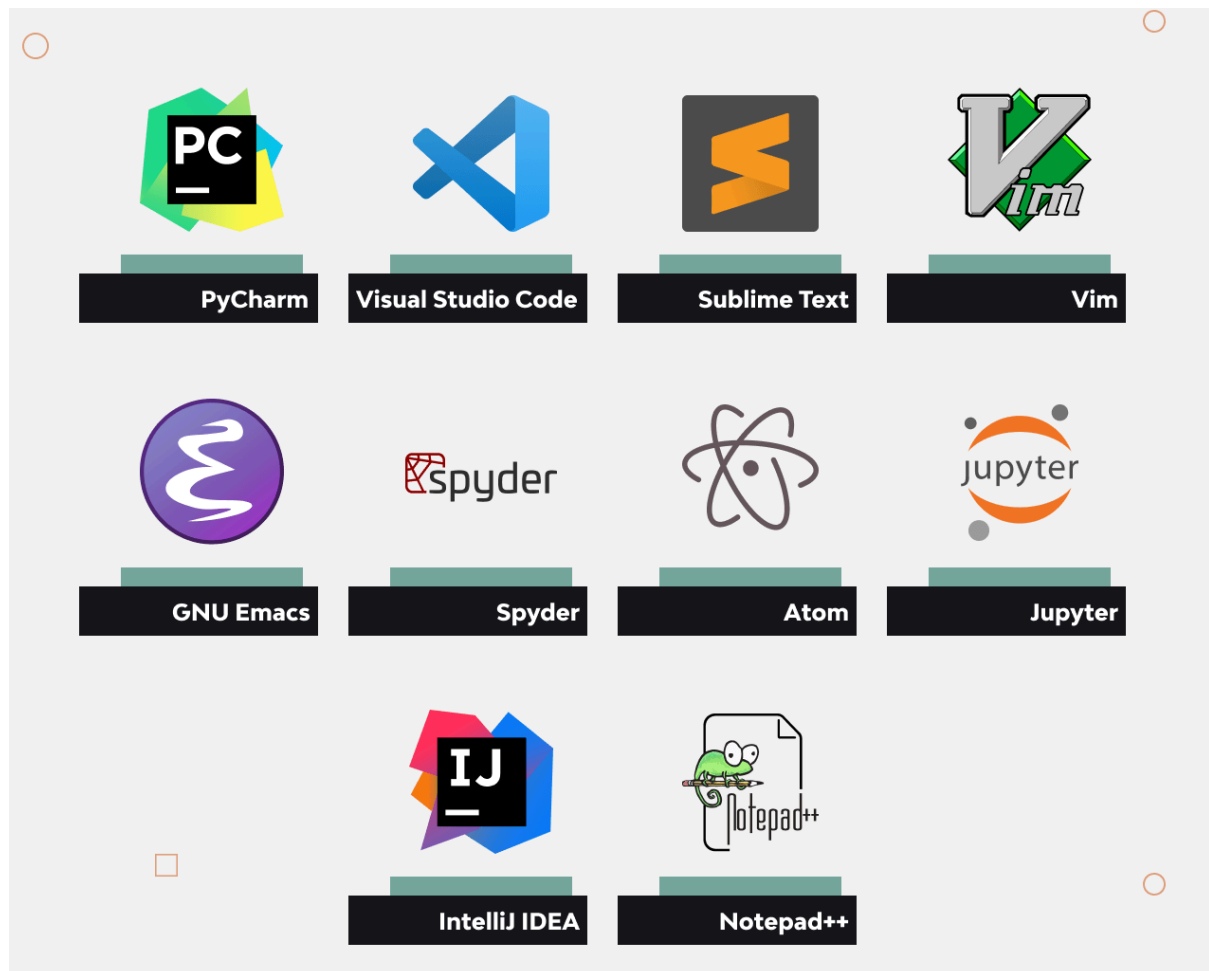
Integrated development environment (IDE) Software

An *Integrated Development Environment (IDE)* is a software application that provides comprehensive facilities to computer programmers for software development.

It typically includes a source code editor, a compiler or interpreter, build automation tools, and debugger functionalities, all integrated into a single graphical user interface (GUI).

There are several IDEs available for Python development, catering to different preferences and needs. We will use JupyterLab as an environment for Jupyter Notebooks in this course.

Here's a list of some of the popular IDEs for Python:



Open Jupyter Notebook

To open Jupyter Notebook on your computer, follow the steps below for your operating system.

Windows

1. Click on the  Windows symbol and type **anaconda** in the search bar.
2. Click on the Anaconda Prompt app.

3. Once the Anaconda Prompt app opens, navigate to the desired folder, using the `cd` command.
4. Once you are in the desired folder, type `jupyter notebook` and press the Enter key.
5. The Jupyter server will start. You should see some server logs printed. You may be prompted to select an application to open Jupyter in Firefox or Chrome.
6. Shortly after, a browser window should open, showing the files and folders located in the folder where you started the Jupyter server.
7. Create a new Jupyter notebook by clicking on New, and then choose the type of Jupyter notebook you want to start, for example Python 3.
8. Your new Jupyter notebook is now ready.

MacOS

1. Open the Terminal app on your Mac by clicking  in the upper-right corner of the menu bar, or pressing Command-Space bar and then typing `terminal`.
2. Once the Terminal app opens, navigate to the desired folder, using the `cd` command.
3. Once you are in the desired folder, type `jupyter notebook` and press the Enter key.
4. The Jupyter server will start. You should see some server logs printed.
5. Shortly after, a browser window should open, showing the files and folders located in the folder where you started the Jupyter server.
6. Create a new Jupyter notebook by clicking on New, and then choose the type of Jupyter notebook you want to start, for example Python 3.
7. Your new Jupyter notebook is now ready.

Linux

1. Open your terminal and navigate to the directory where you would like to save your notebook using the `cd` command.
2. Once you are in the desired folder, type `jupyter notebook` and press the Enter key.
3. The Jupyter server will start. You should see some server logs printed.
4. Shortly after, a browser window should open, showing the files and folders located in the folder where you started the Jupyter server.
5. Create a new Jupyter notebook by clicking on New, and then choose the type of Jupyter notebook you want to start, for example Python 3.
6. Your new Jupyter notebook is now ready.

Exploring the Jupyter Notebook user interface

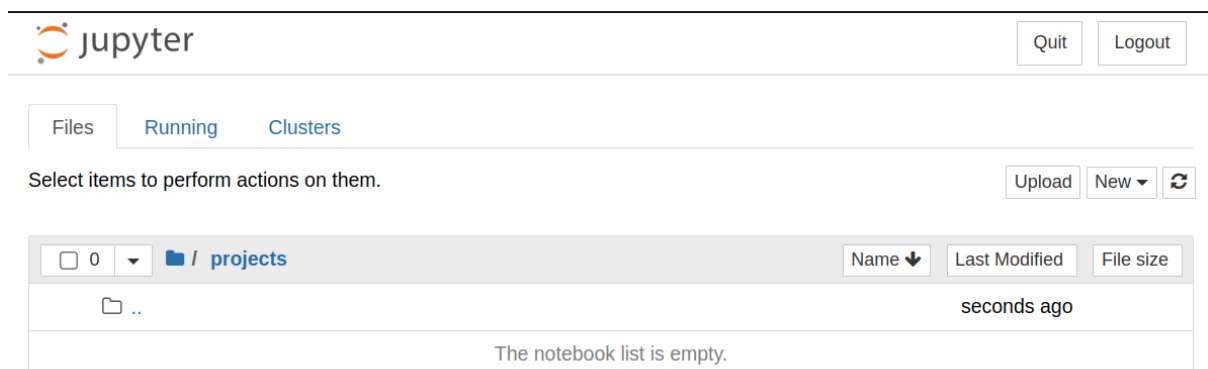
Jupyter Notebook interface

The Jupyter Notebook interface is a web-based environment for creating notebook documents that combine code and output with narrative text, equations and visualisations.

Let's look at the names of the various user interface (UI) elements within the Notebook and the different Notebook modes, as documented on the [official Jupyter website](#).

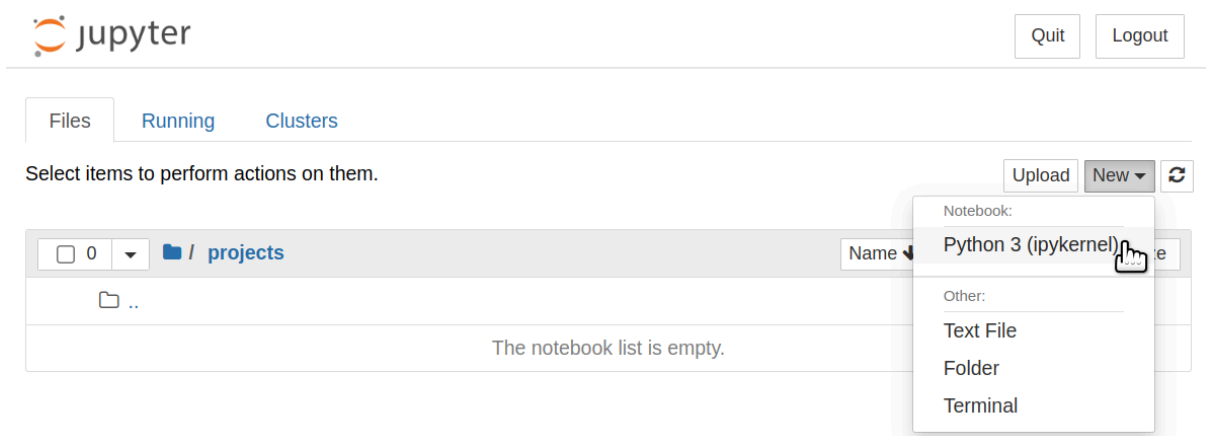
Notebook Dashboard

When you launch **jupyter notebook**, the first page that you encounter is the Notebook Dashboard. It will display a list of all your notebooks (if you have previously created notebooks), or it will show that **The notebook list is empty**.

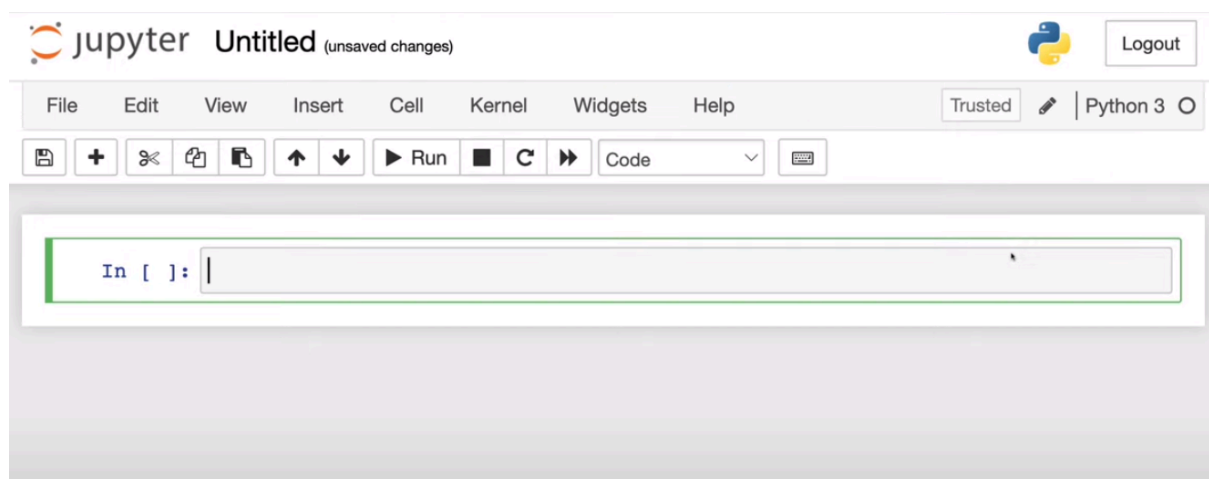


Notebook Editor

Create a new notebook by choosing New > Python 3 or New > Notebook (then select Python 3):



The new notebook will open in the Notebook Editor, like below:



The notebook has three main parts:

- Menu bar where you can access various menu options, such as *File*, *Edit*, and *Help*. For detailed descriptions, see the tutorial [Jupyter Notebook: An Introduction](#) written by Mike Driscoll.
- Toolbar with shortcuts to save the notebook, insert a new cell, format the cell type, or run a cell (see more information on this below).
- Cells where you will type and execute your code.

Let's look at the various elements in this notebook.

Notebook title

At the top of the new notebook, it will show Untitled.

Click on this name and type a new name for your notebook (*my-first-project*), then click Rename.

Cells

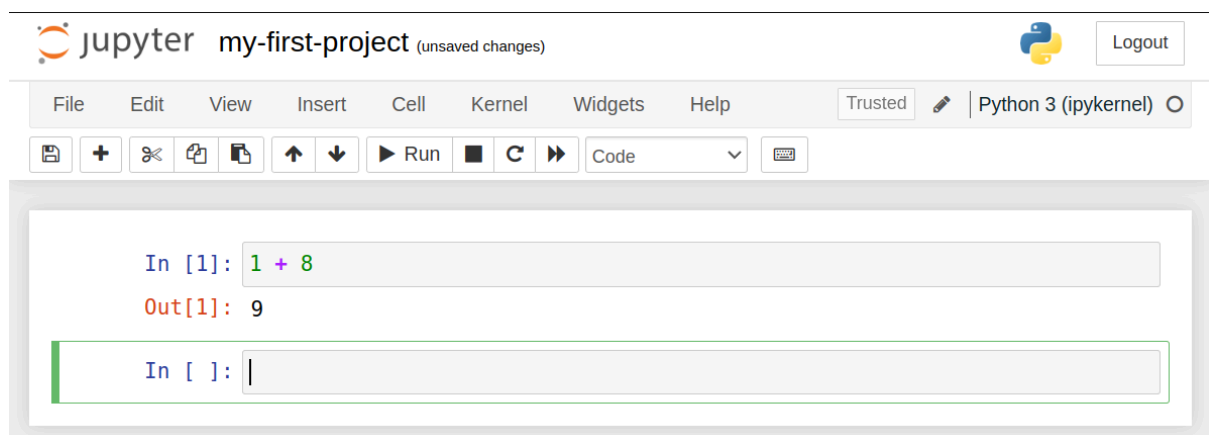
A cell has a numbered input and output area.

There are various cell types to choose from:

- Code: Executable code
- Markdown: Markdown formatted text
- Raw: Plain text

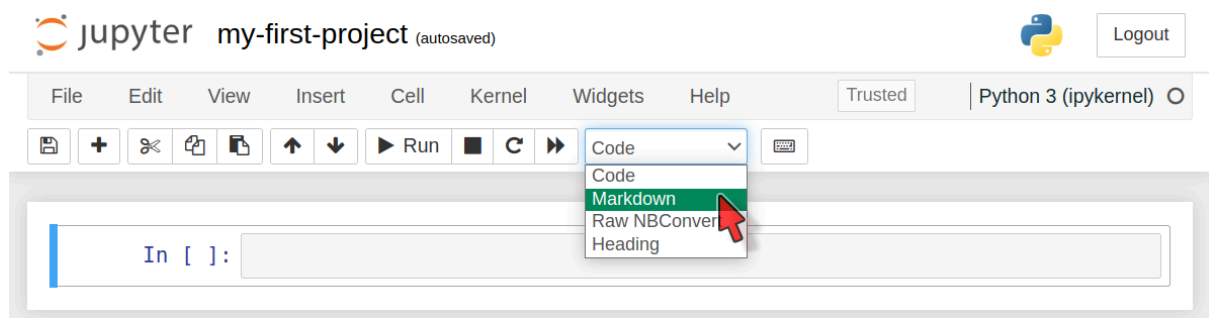
1. Executing code

- Enter code or text into the input area and click ► Run (or **Ctrl + Enter** or **Shift + Enter** for the shortcut) to execute the content of the specific cell.
- When you execute a code cell, that code is sent to an execution kernel, and the result is returned in the output area of the notebook. The output can be values, a dataframe, or a visualisation.
- In some Jupyter Notebook interfaces, like in the example below, you can recognise a code cell by the **In[]**: at the beginning of the cell and output by **Out[]**:
- Pressing the **+** button in the menu bar will add a new cell.
- All your commands, as well as any output, will be saved as part of the notebook file.



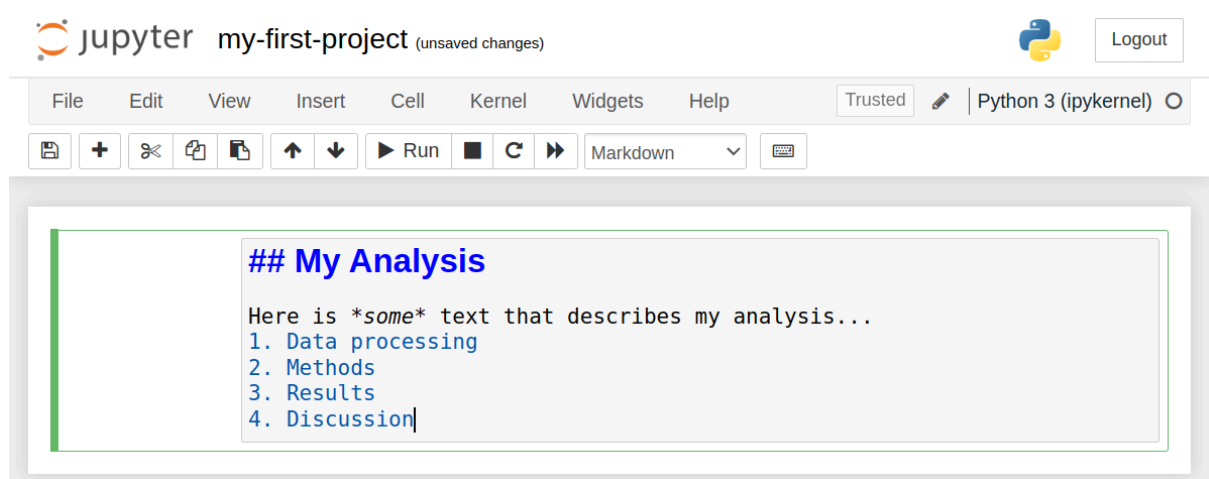
2. Executing markdown or text

- To create a new markdown or raw text cell, click on the dropdown menu with the word Code, and choose Markdown or Raw.

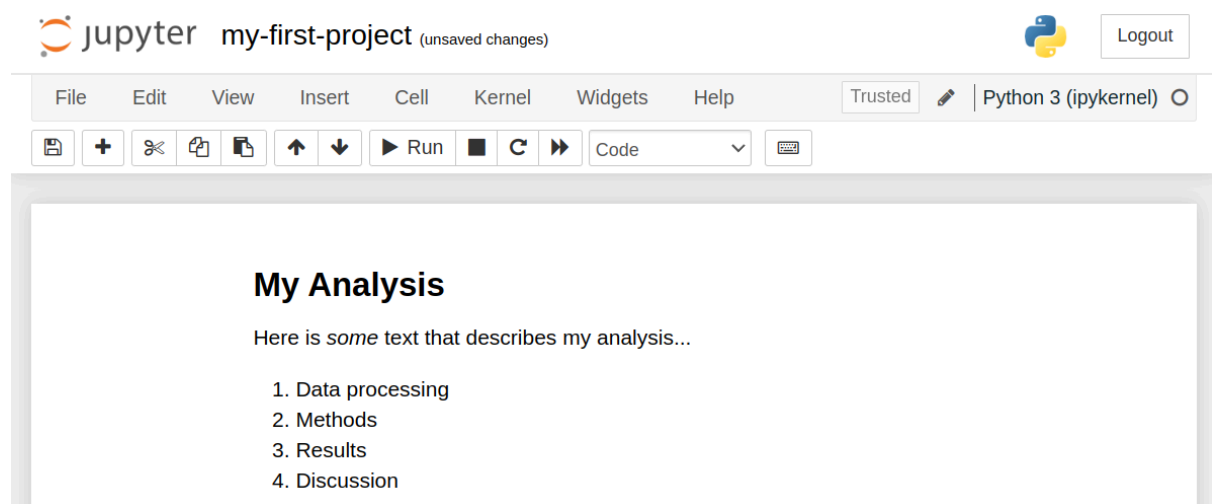


- Enter text into the **input cell** and click ► **Run** (or **Ctrl + Enter** or **Shift + Enter**).
- When executing a markdown or raw text cell, there will not be any output in the output area, but the text will be displayed according to the formatting that you specified.

The following image shows non-rendered markdown (before running the cells):



The following image shows the rendered markdown (after running the cells):



You can do the following [fun tutorial to learn Markdown](#).

If you would like to learn more about the specific elements within the Notebook Editor, you can go through the user interface tour by selecting Help in the menu bar and then selecting User Interface Tour.

Create a reproducible project

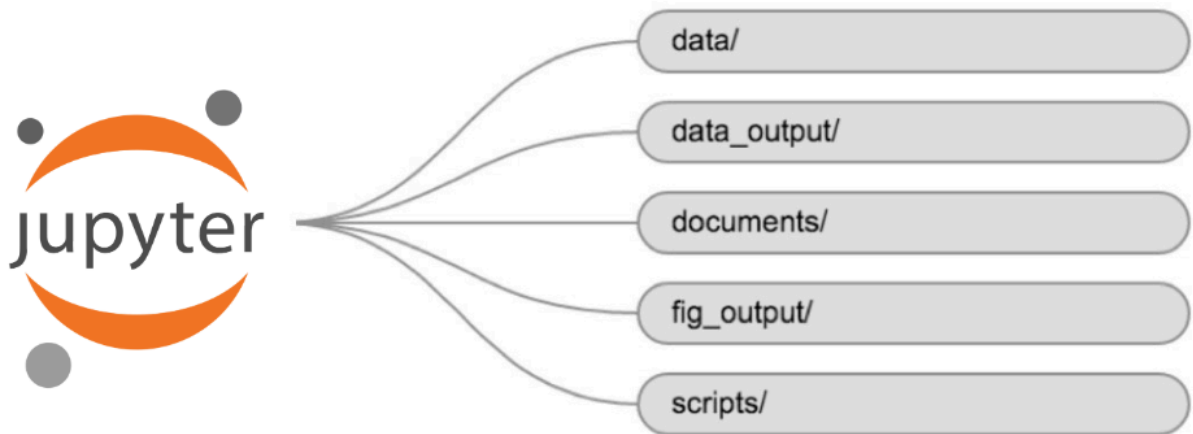
Reproducible projects

What is a reproducible project?

A reproducible project refers to a systematic approach to organising and documenting your code, data, and analysis steps in such a way that it allows others (or even your future self) to easily understand, replicate, and verify your results.

The goal of a reproducible project is to ensure that anyone who interacts with your code and data can obtain the same outcomes, insights, and conclusions as you did, following a well-defined and transparent process.

In general, you might create directories (folders) for scripts, data, and documents. Here are some examples of suggested directories:



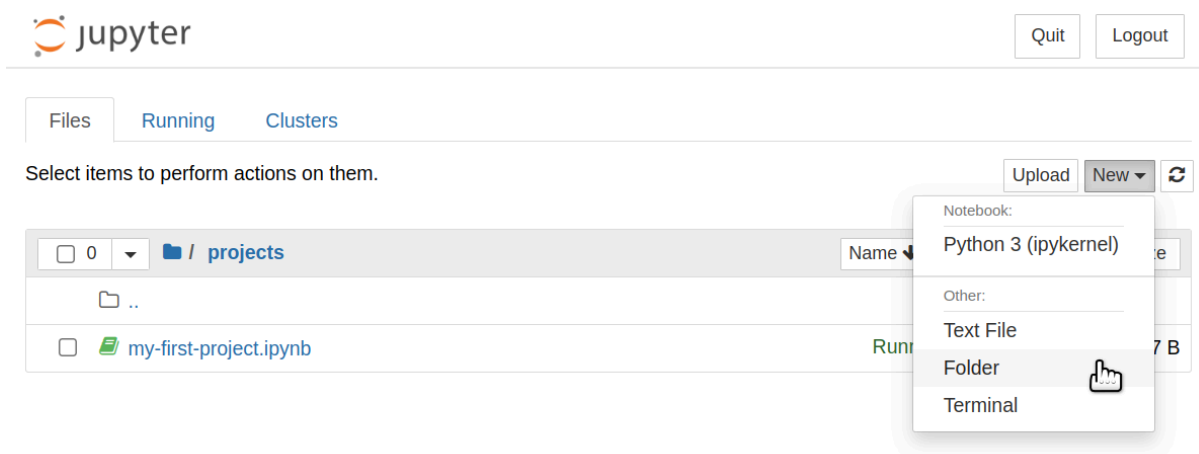
- **data/** Use this folder to store your raw data and intermediate datasets. For the sake of transparency and **provenance**, you should *always* keep a copy of your raw data accessible and do as much of your data cleanup and preprocessing programmatically (i.e., with scripts, rather than manually) as possible.
- **data_output/** When you need to modify your raw data, it might be useful to store the modified versions of the datasets in a different folder.
- **documents/** Used for outlines, drafts, and other text.
- **fig_output/** This folder can store the graphics that are generated by your scripts.
- **scripts/** A place to keep your Jupyter notebooks (scripts) for different analyses or plotting.

You may want to create additional directories or subdirectories depending on your project needs, but these should form the backbone of your working directory 💡.

💡 A working directory is the folder that contains all the subfolders and files of your project. This is where Python will look for files that you ask it to load, and where it will put any files that you ask it to save.

How do I create a reproducible project?

To create a project folder, go to your Jupyter Dashboard and click on New > Folder. This will create a new folder called *Untitled Folder*.

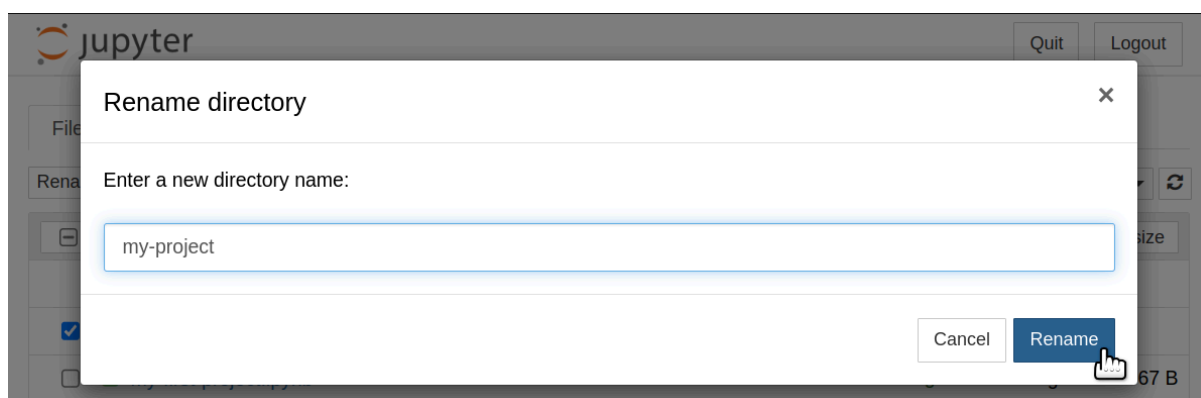


Next, check the checkbox next to *Untitled Folder* and click on Rename at the top of your dashboard.



Type a name for this new folder and click Rename. Make sure it is a meaningful name that will help you remember what the project is about. As with files, we recommend using lowercase letters, no spaces and hyphens to separate words.

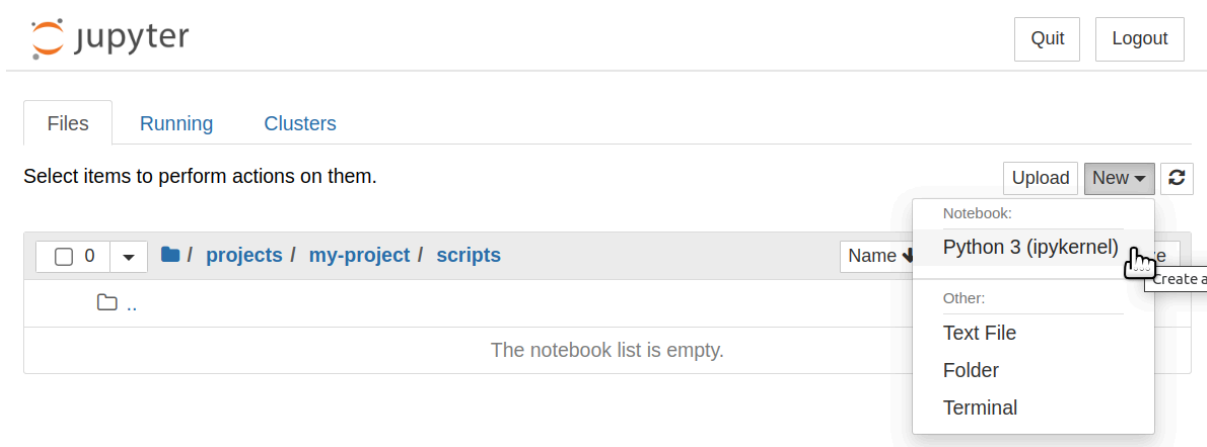
We will call the folder for this project *my-project*.



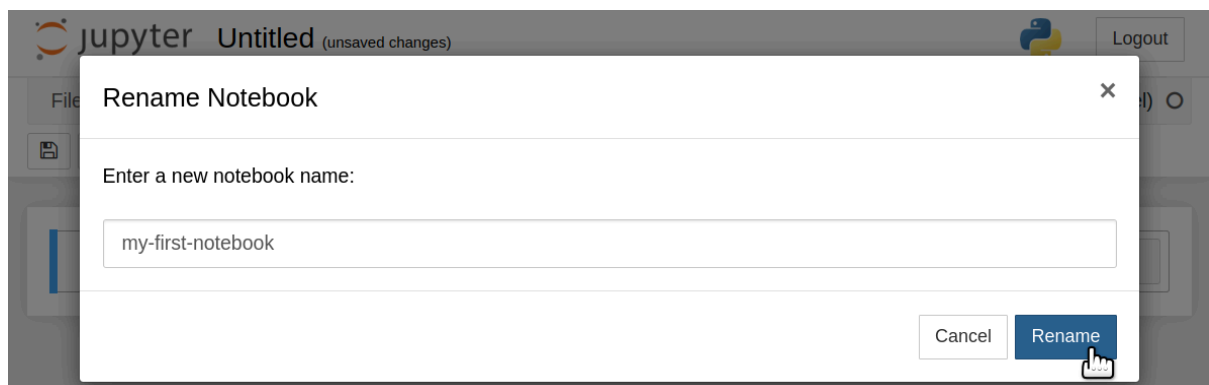
Now it's time to create all the folders that we will need for storing our data, scripts, and documents:

- Click on this new *my-project* folder to go into this folder.
- Click on New > Folder > then rename the folder (e.g. data) according to the previous instructions.
- Repeat this step to create the following three folders: data_output, fig_output, and scripts.

Once you've created all the folders you will need, go into the scripts/ folder. Go ahead and create a new notebook by clicking on New > Python 3.



To rename your notebook, click on Untitled at the top of the notebook and type a name for this notebook (e.g. *my-first-notebook*) and click Rename.



To save changes to your notebook, click the  Save icon at the top of the notebook.



 Note: The notebook will be autosaved while you are working, so you don't need to click Save, but it's good practice to save your work often!

The notebook file is stored in a format called JSON and has the suffix `.ipynb`.

Now you have a reproducible project!

One of the main advantages of using Projects is that you can share this project with colleagues, and they will be able to run the analyses and produce the same results (i.e. the project is reproducible).

Elements in the programming environment

Basic programming elements

The basic elements of programming include:

- input,
- data structures
- variables,
- functions, and
- output.

Input

When you work in Python, you can provide different data structures as input.

This can range from a single value to an entire table with data. You can even use images as input!



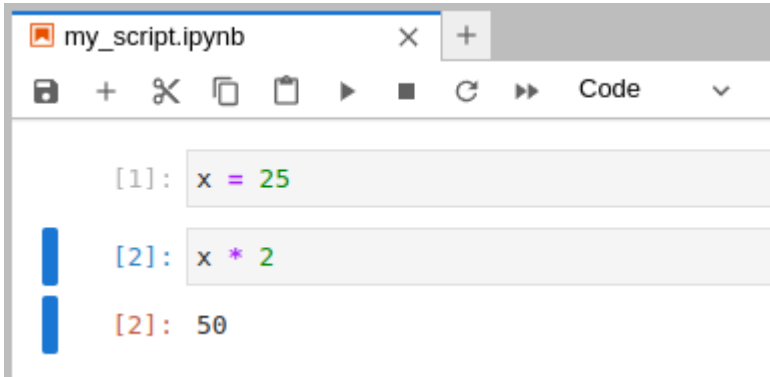
Examples of input:

- A single value: `x = 25`
- A list with numeric values: `y = [10, 20, 30, 40, 50]`
- A tuple with numeric values: `z = (10, 20, 30, 40, 50)`
- A dictionary with key-value pairs: `my_dict = {'Mary': 5, 'John': 7, 'Fatima': 11}`
- A table (also called a dataframe) with multiple columns and rows.

Variables

When you create or import the data, you can save it as a variable and use it to do calculations and/or make visualisations.

 Examples of variables in a Jupyter notebook:



```
[1]: x = 25

[2]: x * 2

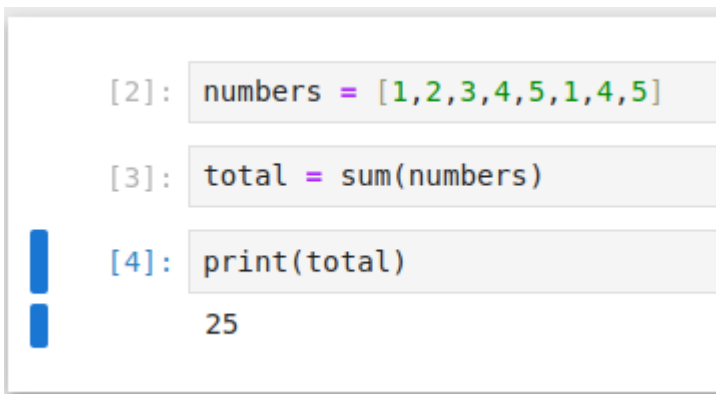
[2]: 50
```

Functions

A function is a set of instructions that you run on the input (i.e. the data) to return the desired output.

For example, the `sum()` function will calculate the sum of multiple values.

 Example of the sum function:



```
[2]: numbers = [1,2,3,4,5,1,4,5]

[3]: total = sum(numbers)

[4]: print(total)

25
```

Output

If you do calculations or you transform the data (input), the output will either be:

- displayed in the output cell;
- saved as a variable; or

- written to an output file (e.g. save the dataframe as a CSV or Excel file, or save the visualisation as a PNG file).

Week 2: Using existing code to save time and be more efficient

1. Understand tidy data
2. Describe the difference between packages and functions
3. Install and use packages and functions
4. Access open data
5. Know how to find out if someone has already published a function that you can re-use

Understand tidy data

What does tidy data look like?

1. Put all your variables in columns - the thing you're measuring, like 'weight' or 'temperature'.
2. Put each observation in its own row. An observation is a single entity (person, animal, etc.).
3. Don't combine multiple pieces of information in one cell. Sometimes it just seems like one thing, but if you can split the data into two separate columns, then do that instead!
4. Leave the raw data raw - don't change it!
5. Export the cleaned data to a text-based format like CSV (comma-separated values) format. This ensures that anyone can use the data, and is required by most data repositories.

Example of untidy data

Date collected	Plot	Species-Sex	Weight
1/9/78	1	DM-M	40
1/9/78	1	DM-F	36
1/9/78	1	DS-F	135
1/20/78	1	DM-F	39
1/20/78	2	DM-M	43
1/20/78	2	DS-F	144
3/13/78	2	DM-F	51
3/13/78	2	DM-F	44
3/13/78	2	DS-F	146

Example of tidy data

Date collected	Plot	Species	Sex	Weight
1/9/78	1	DM	M	40
1/9/78	1	DM	F	36
1/9/78	1	DS	F	135
1/20/78	1	DM	F	39
1/20/78	2	DM	M	43
1/20/78	2	DS	F	144
3/13/78	2	DM	F	51
3/13/78	2	DM	F	44
3/13/78	2	DS	F	146

Difference between packages and functions

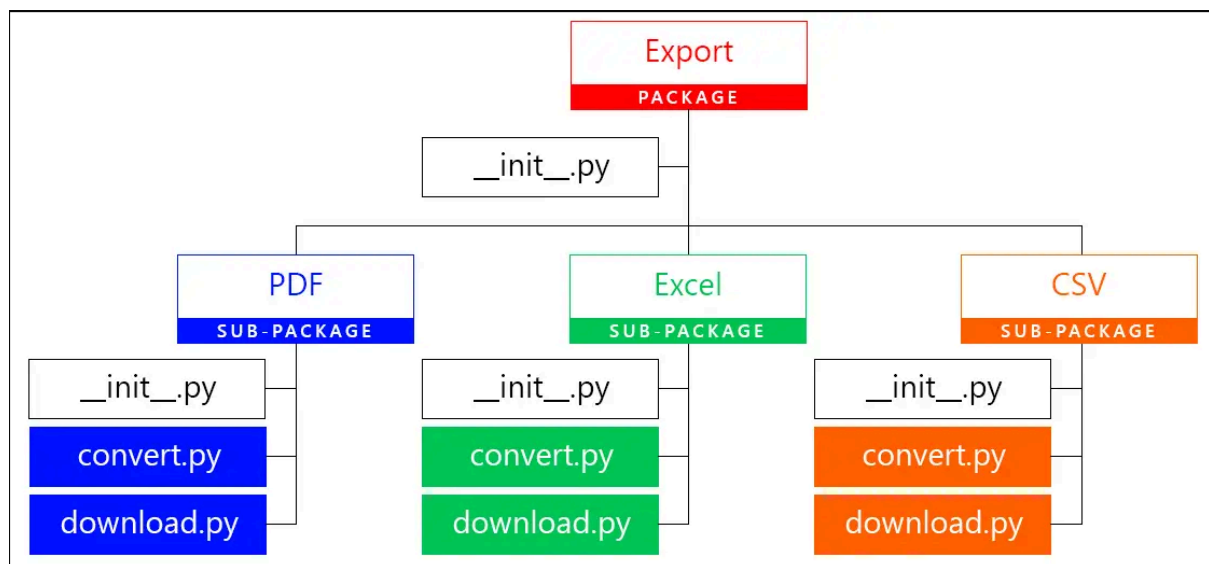
Packages, modules and functions

Packages and modules

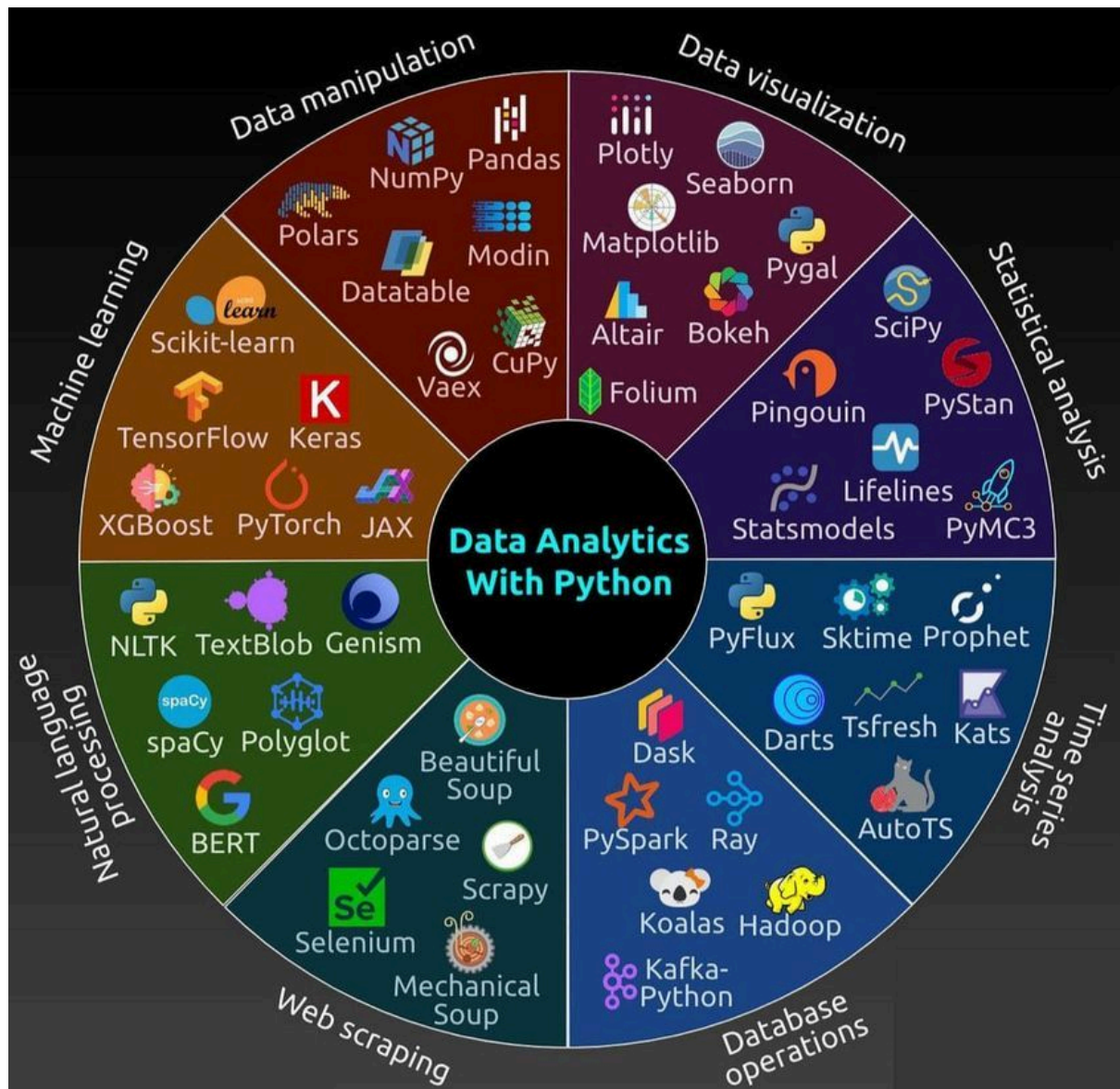
Python **packages** are a collection of Python **modules**. A **module** is a Python file that contains Python code (**functions**) saved with the extension `.py`. It's good to keep related code together in a single module.

A Python **library** contains a collection of modules and packages. This term is often used interchangeably with “Python package” because packages can also contain modules and other packages (sub-packages).

The following diagram, created by Prince Ly Cesar in the [Library vs Module vs Package Python tutorial](#), illustrates the differences between Python packages, sub-packages, and modules (`.py` files).



There are a large number of packages available to use in Python. Here are just a few examples of packages that can be used for data analytics:



By default, the Anaconda distribution comes with over 150 pre-installed Python packages, including the popular packages `numpy`, `pandas`, `scipy`, `sklearn`, `tensorflow`, `pytorch`, and `matplotlib`. A full list of the current packages included, can be found [here](#). However, if you need more packages, you will need to install them. This will be covered in the next lesson.

Functions

A function is a set of instructions that you run on the **input** (i.e., the data) to return the desired **output**.

The 'backbone' of a function looks like this:

function(argument1, argument2, ...)

More about this later...

Install and use libraries and functions

See the following short tutorial on [how to add a library in Jupyter Notebook online](#). For a more comprehensive tutorial, see the [Python Packaging User Guide](#).

Libraries and functions

To use specific functions from a specific library, you need to install the library first. A library is only installed once, but you have to load (import) the library every time you start a new Jupyter session.

It's like buying a pot to make pasta: you only buy the pot once, but you need to take it out of the cupboard every time you want to make pasta!

You can browse the [Python Package Index \(PyPI\) repository](#) of software for Python libraries.

Install a library

To install a new library, you need to open up a *Terminal* (if you are using Mac OSX or Linux), or launch an *anaconda-prompt* (if you are using Windows).

In your terminal window, type the following: `conda install -y -c conda-forge <name>` and press *Enter* to execute this command.

Replace `<name>` with the name of the library, for example `conda install -y -c conda-forge wordcloud`.



Note: This will install the latest version of the library into your conda environment.

Import a library

To import a Python library and make use of the functions, type the following syntax into a code cell in your notebook and run the cell: `import <name>`.

Replace `<name>` with the name of the library, e.g. `import pandas`.

If we want to give the library a nickname to shorten the command, we can add `as <nickname>`, e.g. `import pandas as pd`.

Exercise

1. Install the ... package using the following command:

```
conda install -y -c conda-forge ...
```

2. Now import the library in your script with the following:

```
import ... as ...
```

Call a function

Let's refresh our memories on what a function looks like:

function(argument1, argument2, ...)

- Firstly, you have to type the name of the function, followed by rounded brackets, e.g. `round()`.

 Note: there is no space between the name of the function and the rounded brackets.

- Some functions also take arguments, which are specified within the rounded brackets,

e.g. `round(number = 3.142, ndigits = 2)`.

The arguments change the way the function works,

e.g. if you specify `round(number = 3.142, ndigits = 2)`,

then the `round()` function will round the input value (`number`) to 2 digits.

 You can use the function without typing out the names of the arguments, e.g. `round(3.142, 2)`, but then the arguments should be in the correct order, otherwise, the function might not work as expected or you might get an error. It is best practice to type out the names of the arguments, e.g. `number = 3.142, ndigits = 2`.

If you want to use a function from a specific library, you need to use the syntax `LibraryName.FunctionName`. Adding the library name with a `.` before the function name tells Python where to find the function,

e.g. `pandas.read_csv("data.csv")`.

In the example above, we have imported Pandas as `pd`. This means we don't have to type out `pandas` each time we call a Pandas function, and we can shorten our command to `pd.read_csv("data.csv")`.

Getting help

For more information on how to get help on specific modules and their functions in Jupyter Notebook, have a look at [this page](#).

Access open data

Built-in data sets

There are many datasets available in base Python and in different Python modules. The characteristics of these data sets are very different.

For example, some datasets are time series data, some have only numerical columns, some have numerical as well as factor columns, and some include character columns with the other types of columns, to name a few examples.

Example 1: Seaborn

Seaborn is a package that is available in the Anaconda distribution. By default, *seaborn* is best known for data visualisation, as evident from [this example gallery](#) and [this tutorial page](#), but it also has some great sample datasets that you can use.

To view and load datasets from *seaborn*, execute the following commands in a code cell:

1. To import the `seaborn` module:

```
import seaborn as sns
```

2. To view the datasets in the `seaborn` module:

```
sns.get_dataset_names()
```

3. To load the flights dataset:

```
flights = sns.load_dataset('flights')
```

Example 2: NLTK

(Try this at home 😊)

Another example of a package that is already part of the Anaconda distribution, is *Natural Language Toolkit (NLTK)*. [NLTK](#) is widely used for text processing and analysis tasks. It provides access to various datasets for tasks such as text classification, sentiment analysis, and part-of-speech tagging.

To list the datasets available in NLTK, you can use the `nltk.corpus` module, which provides access to corpora (collections of linguistic data). Here's how you can list the available corpora:

1. Import the [NLTK](#) module:

```
import nltk
```

2. Download the [NLTK](#) book collection which includes various datasets (this step might take a while):

```
nltk.download('book')
```

3. List the available datasets in, for example, the Gutenberg Corpus:

```
gutenberg_files = nltk.corpus.gutenberg.fileids()  
gutenberg_files
```

4. Load a specific text from the Gutenberg Corpus (e.g. Shakespeare's Hamlet) and print the first few characters from the text:

```
hamlet_text = nltk.corpus.gutenberg.raw('shakespeare-hamlet.txt')  
hamlet_text[:200]
```

Re-use published functions

There are thousands of functions in the Python programming language and more functions are created every day.

[Statistics Globe](#), an education platform that provides free programming tutorials, has compiled a [Python Programming Language for Statistics & Data Science tutorial](#). In the tutorials, they provide many examples that you can easily reproduce and adapt for your own needs. Click on a specific video and follow the tutorial with quick examples to learn how the specific function works.

The Python [Graph Gallery](#) also provides multiple examples of visualisations and the code that was used to produce these visualisations.

If you want to re-use this code for your work, you can simply copy and paste the code into your Jupyter Notebook. You will then need to substitute the variable names in the *example code* with the variable names that are in your *dataset*.

For example, the code on the internet shows you how to create a basic scatter plot with the following code: `df.plot("x", "y")`

If you have a dataframe called `surveys`, and you have two continuous variables called `weight` and `height`, you would need to modify the example code as follows:
`surveys.plot("weight", "height")`

Week 3: Working with tabular data

1. Load and inspect tabular data
2. Create a wordcloud
3. Give examples of resources and communities to find help with programming

Load and inspect tabular data

Get the data

Click on [this link](#) to download a dataset to work with.

This data is a subset of the NCHLT English Text Corpora that you can download from the SADiLaR Resource Catalogue: <https://repo.sadilar.org/handle/20.500.12185/301>

This data is a collection, consisting of a clean corpus, lexicon, frequency list and named-entity lists developed during the *NCHLT Text* project.

Upload data to your project

Open the programming software (as we did in [Week 1](#)) then, in your Jupyter dashboard, navigate to the `data/` folder (directory).

- If you have saved the data in this folder, you should see the file that you downloaded.
- Alternatively, click `Upload` and browse for the file, then select the file and click `Open`.

Load the pandas library and the data

One of the most useful Python libraries for working with tabular data is the [Python Data Analysis Library](#) (a.k.a. Pandas). The [pandas library](#) provides data structures, produces high-quality plots with [matplotlib](#) and integrates nicely with other libraries.

Open the notebook file ([.ipynb](#)) that we created previously and execute the following steps:

1. Import the pandas library:
`import pandas as pd` (remember to run the code cell)
2. Load the data and save it to memory by assigning the dataframe to a variable:
`data = pd.read_csv("../data/<name>.csv")` and replace `<name>` with the filename.

Inspect data

There are several commands we can use to inspect the data. You can view attributes or methods.

Viewing attributes 💡

- `data.columns` -> shows an index of all the column (variable) names
- `data.dtypes` -> shows the variables and data types of the dataframe
- `data.shape` -> shows the dimensions (number of rows and columns) of the dataframe

Viewing methods 💡

- `data.head()` -> displays the first few rows of the data
- `data.tail()` -> displays the last few rows of the data
- `type(data)` -> indicates the structure of the data
- `data.describe()` -> shows descriptive statistics of all numeric columns
- `data["frequency"].describe()` -> shows descriptive statistics of only the "frequency" column

💡 Note: there are no parentheses after the attribute, but there are parentheses after the methods.

Create a word cloud

Now that you have downloaded and loaded your data into a script, you can apply many other functions to this data frame object.

One fun thing that you can do with word frequency data like this, is create a word cloud.

Create a word cloud

We will need quite a few lines of code to create this word cloud.

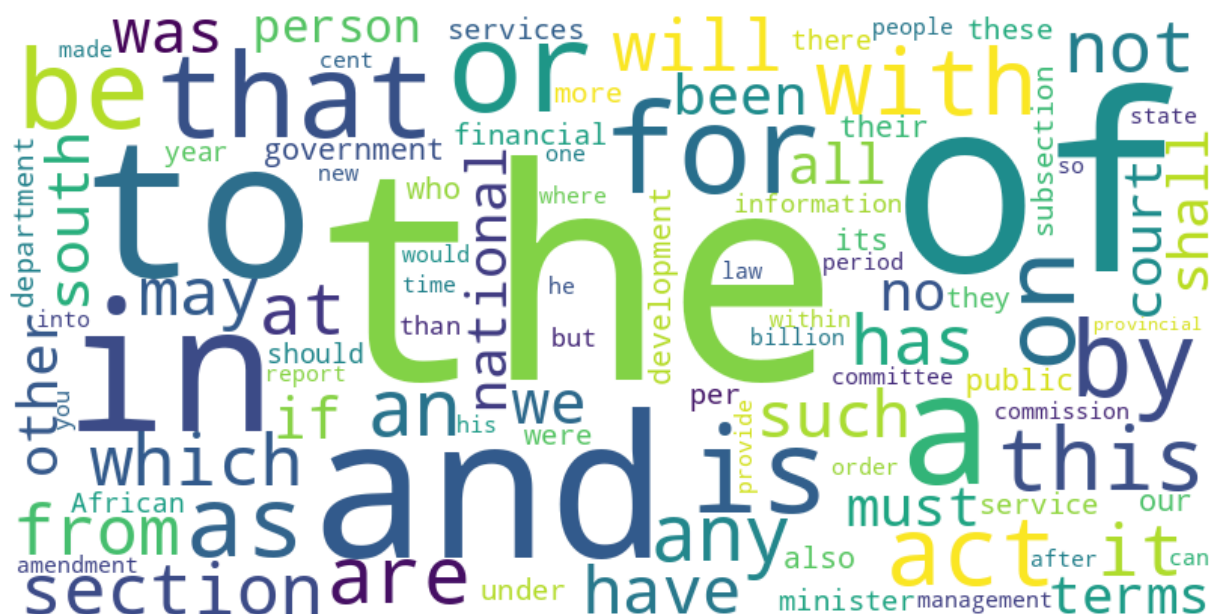
Here are the steps:

1. First load all of the necessary libraries, including `pandas`, `wordcloud` and `matplotlib` .
2. Load the data from the CSV file and create a DataFrame object using the `pd.read_csv()` function from the `pandas` library.
3. Create a dictionary from the data frame columns using the `dict()` and `p()` functions with the specific columns as arguments.
4. Create a WordCloud object using the `WordCloud()` and `generate_from_frequencies()` functions from the `wordcloud` library.
5. Save the image as a PNG file with the `to_file()` function from the `wordcloud` library.
6. You can also use the `matplotlib` library to display the WordCloud in your script/document. For this, you will probably need the `figure()`, `imshow()`, `axis()` and `show()` functions.

The file with an example of the code for the above steps can be downloaded [here](#). You will need to replace the file paths and names to match your own.

Output

In the end, your word cloud image will look like this:



This is of course not the only way to create a word cloud and save the image file - there are many other methods and libraries that you can use.

Resources

Here are some other resources that you might find useful:

- The [wordcloud](#) library [documentation](#) and a [tutorial](#).
- The [matplotlib](#) library [documentation](#) and [cheatsheets](#).

Resources and communities to find help with programming

Interactive learning

If you are interested in learning Python in an interactive way, try [futurecoder](#) or [educative](#) online for free.

Videos, webinars and blogs

- If you prefer learning through instruction, have a look at the [free videos and webinars](#) provided by [scrimba](#).
- If you prefer to read, look at [learnpython.com](#).

Examples with visualisation and code

- [Python Graph gallery](#)

Cheat sheets

View the following [cheat sheets](#) for free (online) or print the [Python 3 cheat sheet](#) and keep it next to your computer to remind you of the various functions that you can use for data wrangling and visualisation.

Books

There are many [open source books](#) available. I would suggest that you choose the "Beginner" option (top menu bar) and move onto the more advanced topics from there.

Communities

Join the following communities or forums:

- ESCALATOR Workspace:
 - To join the workspace, click [here](#).
 - To go to the workspace if you have already joined, go to dhcssza.slack.com.
- Ask questions about anything that is related to Python on the [Python community forum](#).

Week 4: Best practices in programming

- Give examples of best practices in programming
- Recognise good naming conventions
- Make code re-useable and decipherable (commenting)
- Understand the first steps to reading error messages
- Identify possible next steps to continue learning to programme

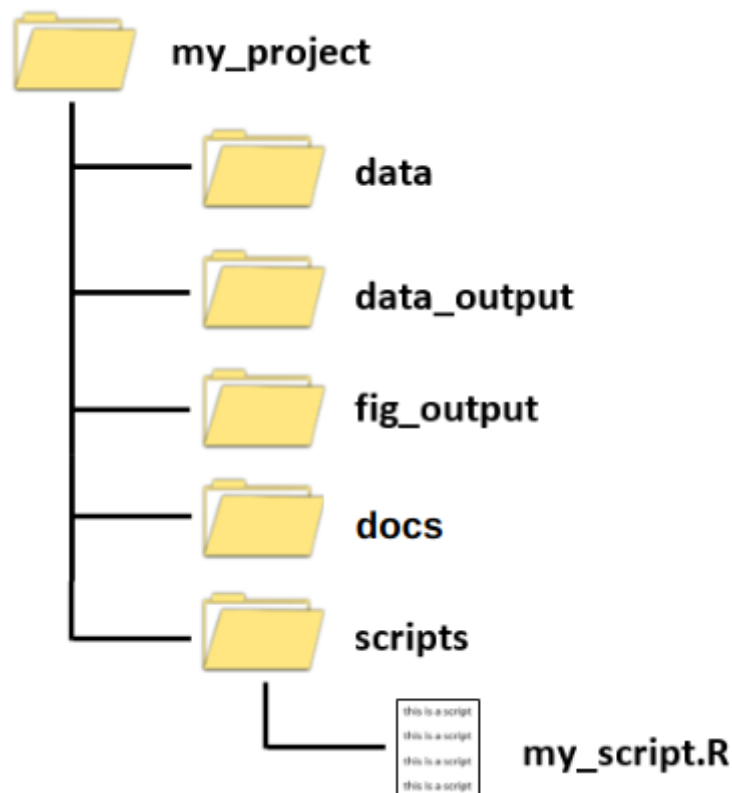
Examples of best practices in programming

File organisation

Create a reproducible project and organise files in a sensible way within the project:

- **data**: this folder contains the raw data and should never be modified.

- **data_output**: this folder stores any intermediate and final datasets that were generated via code during analyses.
- **fig_output**: all the visualisations produced during the analyses are stored in this folder.
- **docs**: store any documents related to your project, such as notes, manuscripts, etc.
- **scripts**: save all the scripts with code in this folder.



Code organisation

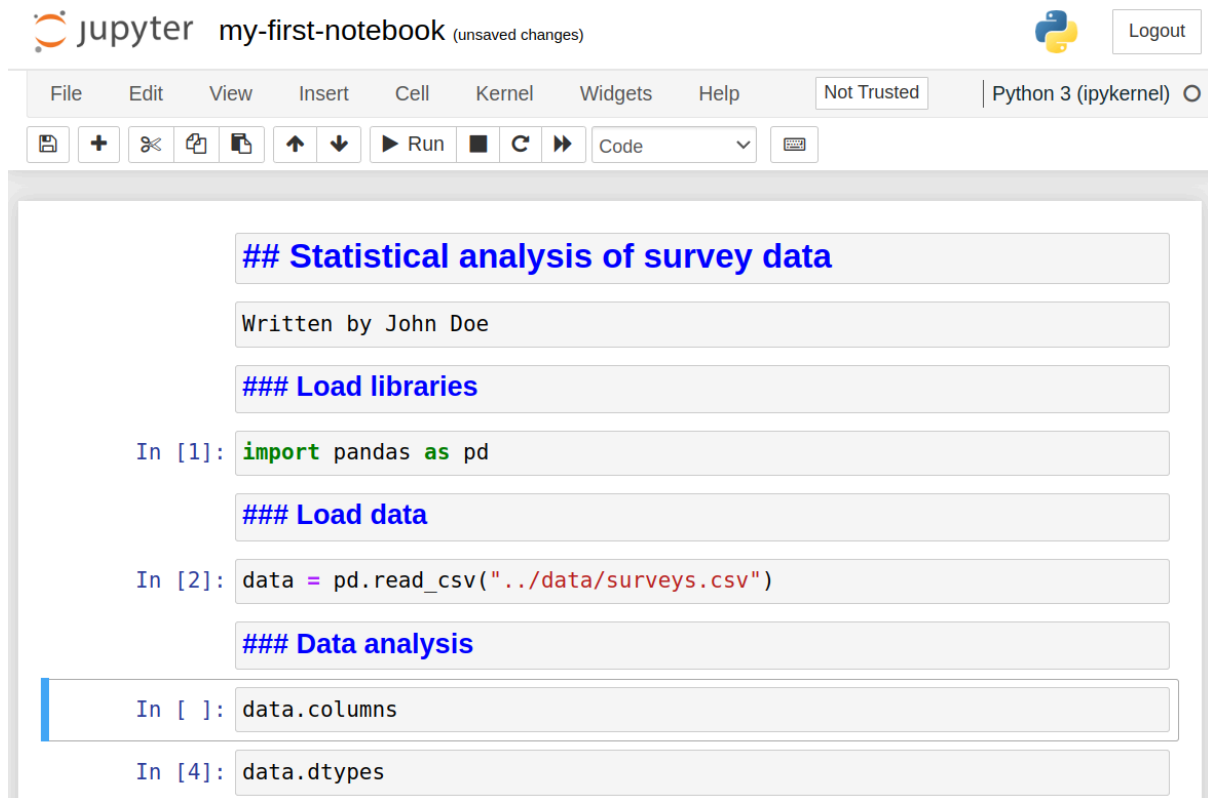
Organise code within each script/notebook:

- Start each file with a comment saying who wrote it and when, what it contains, and how it fits into the larger program.
- Then load all required libraries (each on a separate line).
- Then source (load) required files (if any).
- Then your code starts.
- If you have a lot of code or different analyses, break the code up into separate files (generally <2000–3000 lines).

- Use comments to mark off sections of code, and also to explain the purpose of the various code blocks.

Comments should address the “why?” and not the “what?”.

For example:



Another example:

Data analysis

Step 1: Inspect the data

```
In [ ]: data.columns
```

```
In [ ]: data.dtypes
```

Step 2: Descriptive statistics

```
In [ ]: data["weight"].describe()
```

Step 3: Plot the data

```
In [ ]: import matplotlib.pyplot as plt  
plt.hist(data["weight"])  
plt.show()
```

Relative paths

Always use relative paths in your scripts to ensure it can run successfully on any other computer.

For example:

😞 `mydata = read_csv("C:/Users/John/PhD/project/data_raw/surveys.csv")`

😄 `mydata = read_csv("project/data_raw/surveys.csv")`

Assignment

Use `=` for assignment.

For example:

😄 `mydata = pd.read_csv("surveys.csv")`

Naming conventions

- Generally, variable names should be nouns and function names should be verbs. For example, `country` can be used as a variable name in your dataset, and `calculate_birth_rate` can be used as a function name.

- Use names that are concise and meaningful.
For example, `pop_counts` (for population counts) is better than using a single letter such as `n`.
- Names of variables or functions should not start with numbers.
- It's generally easier to write everything in lowercase, but you can also use periods (e.g. `plot.new`), underscores (e.g. `plot_new`), or camel case (e.g. `PlotNew`) to name variables or functions.

More about this later...

Code review

It's always a great idea to have someone else review your code.

This is easier when you have created a reproducible project and used the above best practices.

Others

Other best practices also include using tabs instead of space for indentation, limiting the lines to a maximum length (e.g. 79 characters), and many more.

If you want to read more about a good Python coding style, have a look at the [PEP 8 style guide for Python code](#).

Recognise good naming conventions

Naming conventions

Naming objects

There's no single correct way to name objects, but the following guidelines are recommended:

- Use only lowercase letters and numbers.
- Use names that are concise and meaningful (this is not easy!).
- Where possible, avoid re-using names of common functions and variables. This will confuse the readers of your code.
- If you find yourself attempting to cram data into variable names (e.g. `model_2018`, `model_2019`, `model_2020`), consider using a list or dataframe instead.

Examples:

- alllowercase: `adjustcolor`
- period.separated: `plot.new`

- underscore_separated: `numeric_version`
- lowerCamelCase: `addTaskCallback`
- UpperCamelCase: `SignatureMethod`

💡 Keep in mind: using all lowercase letters might be difficult to read, the period (.) and underscore (_) are used in many function names instead. Camel case is readable but requires you to use the Shift button excessively. Choose one option and use it consistently.

Naming functions

According to the [style guide by Hadley Wickham](#), variable and function names should be lowercase and you should use an underscore (_) to separate words within a name. Generally, function names should be verbs.

As with objects, strive for names that are concise and meaningful (this is not easy!).

Where possible, avoid using names of existing functions and variables. Doing so will confuse the readers of your code.

Naming output files

For instance, when naming an Interactive Python Notebook file, the file name should end with `.ipynb` and, of course, be meaningful.

Examples:

😞 `foo.ipynb`

😊 `predict_ad_revenue.ipynb`

📖 For more details, you can also have a look at [Google's Python Style Guide](#) or the [PEP 8 style guide for Python code](#).

Make code re-usable and decipherable

Good versus bad coding practice

This example needs some improvement...

jupyter my-first-notebook Last Checkpoint: Last Thursday at 14:42 (unsaved changes) Python 3 (ipykernel) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted

Run Code

```

In [1]: import pandas as pd

In [2]: data = pd.read_csv("../data/surveys.csv")

In [ ]: data.columns

In [ ]: data.dtypes

In [ ]: data["weight"].describe()

In [ ]: import matplotlib.pyplot as plt
        plt.hist(data["weight"])
        plt.show()

```

This is better...

jupyter my-first-notebook Last Checkpoint: Last Thursday at 14:42 (autosaved) Python 3 (ipykernel) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted

Run Code

```

## Statistical analysis of survey data

Written by John Doe

### Load libraries

In [1]: import pandas as pd

### Load data

In [2]: data = pd.read_csv("../data/surveys.csv")

### Data analysis

Step 1: Inspect the data

In [ ]: data.columns

In [ ]: data.dtypes

Step 2: Descriptive statistics

In [ ]: data["weight"].describe()

Step 3: Plot the data

In [ ]: import matplotlib.pyplot as plt
        plt.hist(data["weight"])
        plt.show()

```

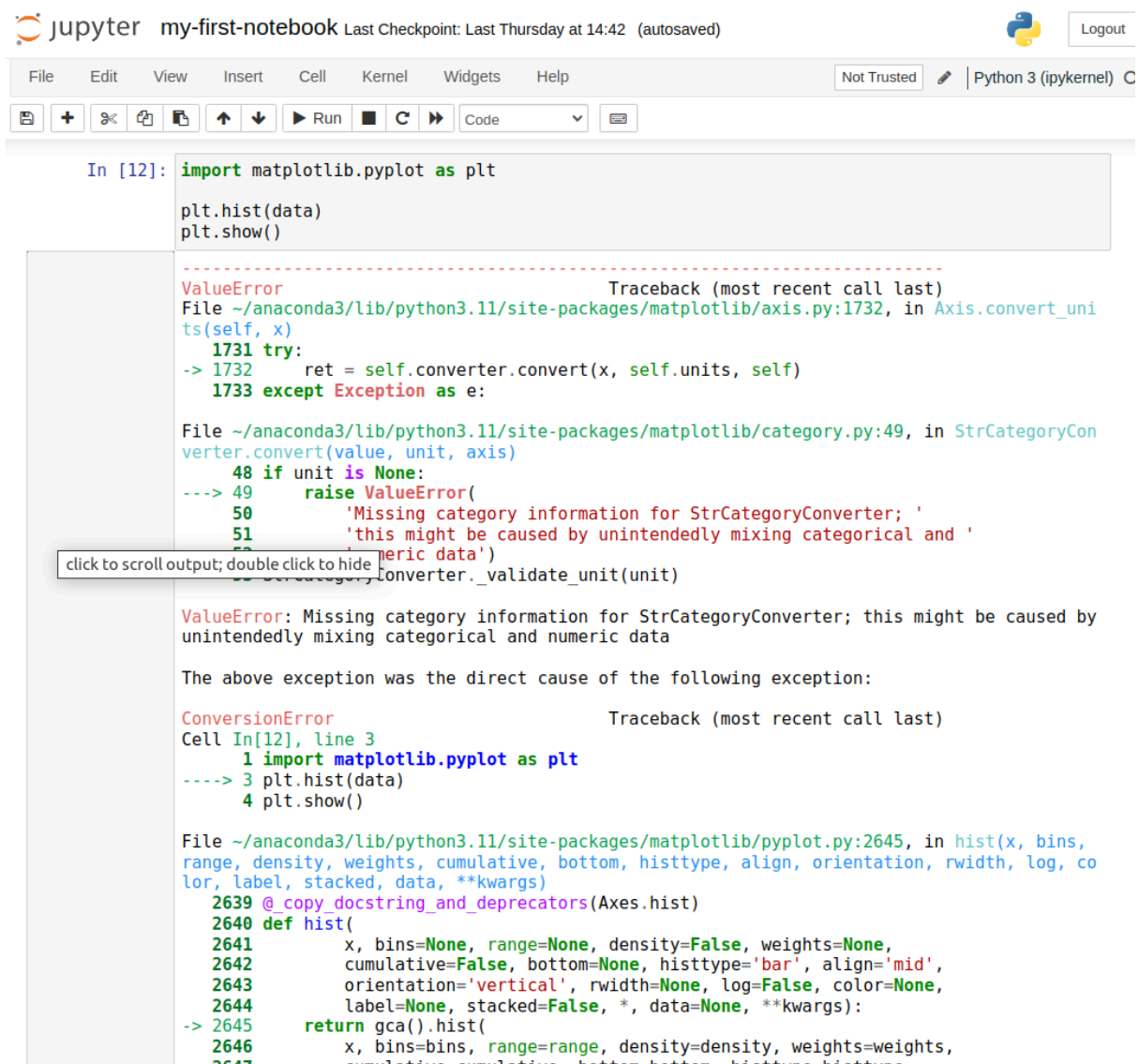
Reading error messages

Errors in code

Don't get discouraged if your code doesn't run immediately! Error messages are common when programming, and fixing errors is part of any programmer's daily work.

Often, the problem is a small typo in a variable name or a missing parenthesis.

Another example is if you forgot to specify which variable you want to inspect or visualise, like in the example below:



```
In [12]: import matplotlib.pyplot as plt

plt.hist(data)
plt.show()

-----
ValueError                                Traceback (most recent call last)
File ~/anaconda3/lib/python3.11/site-packages/matplotlib/axis.py:1732, in Axis.convert_units(self, x)
    1731 try:
-> 1732     ret = self.converter.convert(x, self.units, self)
    1733 except Exception as e:

File ~/anaconda3/lib/python3.11/site-packages/matplotlib/category.py:49, in StrCategoryConverter.convert(value, unit, axis)
     48 if unit is None:
--> 49     raise ValueError(
    50         'Missing category information for StrCategoryConverter; '
    51         'this might be caused by unintentionally mixing categorical and '
    52         'numeric data')
    53     converter._validate_unit(unit)

ValueError: Missing category information for StrCategoryConverter; this might be caused by
unintentionally mixing categorical and numeric data

The above exception was the direct cause of the following exception:

ConversionError                            Traceback (most recent call last)
Cell In[12], line 3
      1 import matplotlib.pyplot as plt
----> 3 plt.hist(data)
      4 plt.show()

File ~/anaconda3/lib/python3.11/site-packages/matplotlib/pyplot.py:2645, in hist(x, bins, range, density, weights, cumulative, bottom, histtype, align, orientation, rwidth, log, color, label, stacked, data, **kwargs)
    2639 @_copy_docstring_and_deprecators(Axes.hist)
    2640 def hist(
    2641     x, bins=None, range=None, density=False, weights=None,
    2642     cumulative=False, bottom=None, histtype='bar', align='mid',
    2643     orientation='vertical', rwidth=None, log=False, color=None,
    2644     label=None, stacked=False, *, data=None, **kwargs):
-> 2645     return gca().hist(
    2646         x, bins=bins, range=range, density=density, weights=weights,
    2647         cumulative=cumulative, bottom=bottom, histtype=histtype,
```

Therefore, to fix the code above, we need to specify the variable (column) that we want to visualise: `plt.hist(data["weight"])`.

If you can't fix an error yourself, start by googling it. Some error messages are too generic to diagnose a problem. In that case, it might help to include the name of the function or library you're using in your query.

Asking for help

If your Google search is unsuccessful, you may want to ask other Python users for help. There are different places where you can ask for help.

For instance, during workshops like these, don't hesitate to talk to the other participants, compare your answers, and ask for help.

You might also be interested in organising regular meetings following the workshop to keep learning from each other.

If you have a friend or colleague with more experience than you, they might be able and willing to help you.

Besides the above, there are a few places on the internet that provide help:

- Join the ESCALATOR Slack workspace and ask questions there:
 - To join the workspace, click [here](#).
 - To go to the workspace if you have already joined, go to dhcssza.slack.com.
- [Stack Overflow](#): Many questions have already been answered, but the challenge is to use the right words in your search to find them. If your question hasn't been answered before and is well crafted, chances are you will get an answer in less than 5 min. Remember to follow their guidelines on [how to ask a good question](#).
- You can also try to contact the library author directly, by emailing them or opening an issue on the code repository (e.g., on GitHub).

Continue learning programming

Additional resources

The material we cover during this workshop will give you a taste of how you can use Python to analyse data for your own research. However, to do advanced operations such as cleaning your dataset, using statistical methods, or creating beautiful graphics you will need to learn more.

The best way to become proficient and efficient at Python, as with any other tool, is to use it to address your actual research questions. As a beginner, it can feel daunting to have to write a script from scratch, and given that many people make their code available online, modifying existing code to suit your purpose might get first hands-on

experience using Python for your own work and help you become comfortable eventually creating your own scripts.

Workshops

If you feel that you learn best through instruction, then attending a hands-on [Carpentries workshop](#) is a good option.

Interactive learning

If you are interested in learning Python in an interactive way, try [futurecoder](#) or [educative](#) online for free.

Videos, webinars and blogs

- If you prefer learning through instruction, have a look at the [free videos and webinars](#) provided by scrimba.
- If you prefer to read, look at [learnpython.com](#).

Examples with visualisation and code

- [Python Graph gallery](#)

Cheat sheets

View the following [cheat sheets](#) for free (online) or print the [Python 3 cheat sheet](#) and keep it next to your computer to remind you of the various functions that you can use for data wrangling and visualisation.

Books

There are many [open source books](#) available. I would suggest that you choose the "Beginner" option (top menu bar) and move onto the more advanced topics from there.

Communities

Join the following communities or forums:

- ESCALATOR Workspace:
 - To join the workspace, click [here](#).
 - To go to the workspace if you have already joined, go to [dhcssza.slack.com](#).
- Ask questions about anything that is related to Python on the [Python community forum](#).

